Introduction to Decision Control in C Programming

What is Decision Control?

- Decision control statements allow a program to take decisions based on certain conditions.
- They help the program to execute certain blocks of code depending on whether the condition is true or false.
- This is essential for making programs dynamic and responsive.

Types of Decision Control Statements in C

1. **if statement**
2. **if-else statement**
3. **if-else-if ladder**
4. **switch statement**

1. if statement :

The `if` statement in C is a control flow statement used for conditional execution. It allows a program to execute a specific block of code only when a given condition is true (evaluates to a non-zero value). If the condition is false (evaluates to zero), the code block is skipped.

Syntax:

```
if(condition) {
    // statements to execute if condition is true
}
```

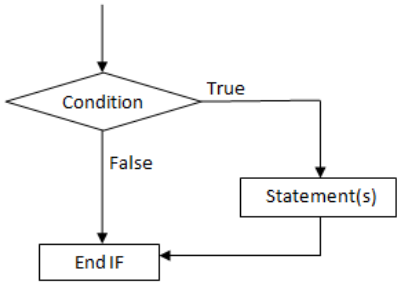


fig: Flowchart for if statement

- The block executes only if the condition evaluates to true.
- If the condition is false, the block is skipped.

Example:

```
int x = 10;
if (x > 5) {
    printf("x is greater than 5\n");
}
```

f Statement Example

```
#include <stdio.h>

int main() {
    int num = 10;

    if (num > 5) {
```

```
        printf("Number is greater than 5\n");
    }

    return 0;
}
```

How it works:

- The condition `num > 5` is evaluated.
- If true (which it is, since 10 > 5), the code inside the `if` block executes.
- If false, the program skips the block.

2. if-else statement:

The `if-else` statement in C is a fundamental control flow structure used for decision-making. It allows a program to execute different blocks of code based on whether a given condition evaluates to true or false.

Syntax:

```
if(condition) {
    // executes if condition is true
} else {
    // executes if condition is false
}
```

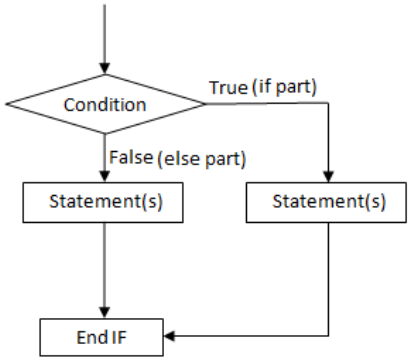


fig: Flowchart for if ... else statement

- Provides two paths: one for true, one for false.

Example:

```
int x = 3;
if (x % 2 == 0) {
    printf("x is even\n");
} else {
    printf("x is odd\n");
}
```

if-else Statement Example

```
#include <stdio.h>

int main() {
    int num = 4;

    if (num % 2 == 0) {
        printf("Number is even\n");
    } else {
```

```
        printf("Number is odd\n");
    }

    return 0;
}
```

How it works:

- Condition checks if `num` is divisible by 2.
- If true, it prints "even".
- Otherwise, it prints "odd".
- Exactly one of the two blocks executes.

3. if-else-if ladder:

The if-else-if ladder statement in C is a control structure used to handle multiple conditions sequentially. It allows the program to evaluate several test expressions and execute different blocks of code based on the first condition that evaluates to true.

- Used when multiple conditions need to be checked sequentially.
- The first true condition block executes; others are skipped.

Syntax:

```
if (condition1) {
    // block1
} else if (condition2) {
    // block2
} else {
    // block3
}
```

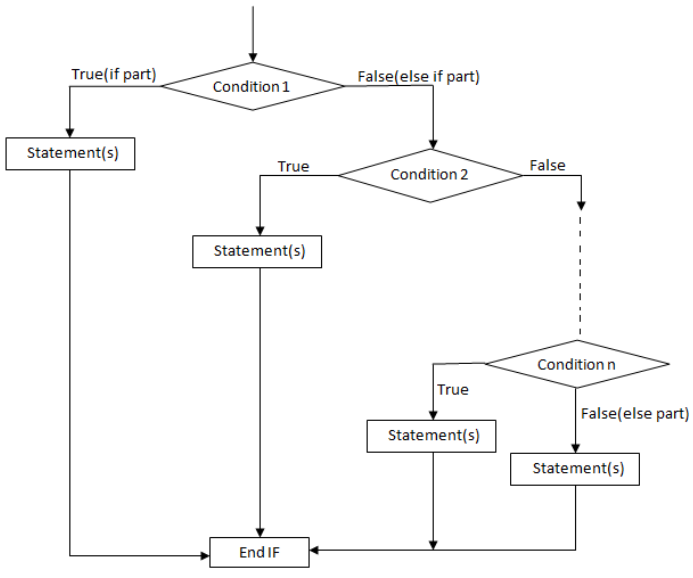


fig: Flowchart for if ... else if ... else statement

Example:

```
int marks = 85;
if (marks >= 90) {
    printf("Grade A\n");
} else if (marks >= 75) {
    printf("Grade B\n");
} else {
    printf("Grade C\n");
```

```
}
```

if-else-if Ladder Example

```
#include <stdio.h>

int main() {
    int marks = 78;

    if (marks >= 90) {
        printf("Grade A\n");
    } else if (marks >= 75) {
        printf("Grade B\n");
    } else if (marks >= 50) {
        printf("Grade C\n");
    } else {
        printf("Fail\n");
    }

    return 0;
}
```

How it works:

- Conditions are checked from top to bottom.
- First true condition's block executes.
- Others are skipped, so only one block runs.
- If none is true, the `else` block runs.

4. Nested if statements

When if statement is kept inside another if statement, it is called nested if statement. Nested if statements are used if there is a sub condition to be tested. The depth of nested if statements depends upon the number of conditions to be checked.

```
if (condition 1)
{
    statements;
    if (sub condition 1)
    {
        statements;
    }
    statements;
}
else if (condition 2)
{
    statements;
    if (sub condition 2)
    {
        statements;
    }
    statements;
}
... ... ...
... ... ...
else
{
    statements;
    if (sub condition n)
    {
        statements;
    }
    statements;
}
```

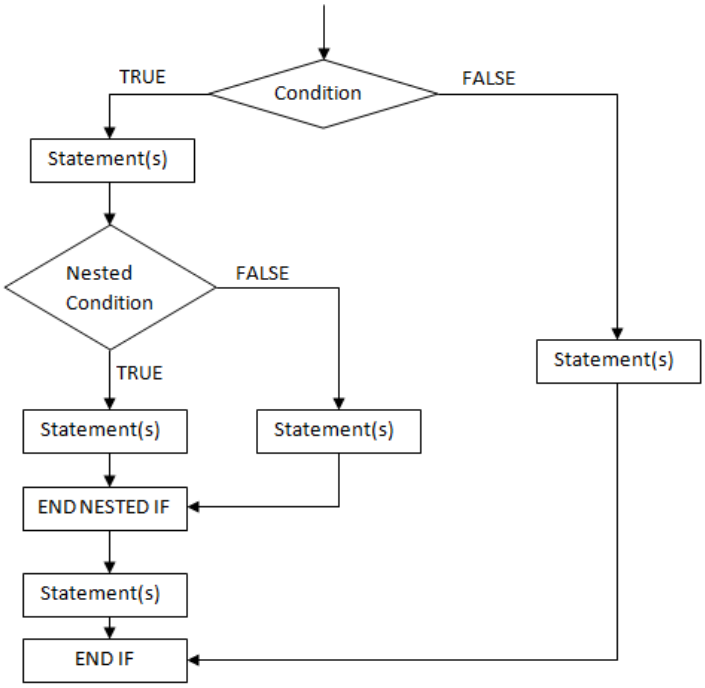


fig: Flowchart for nested if statement

```
#include<stdio.h>
int main()
{
    int n;
    printf("Enter a number:");
    scanf("%d",&n);
    if(n<100)
    {
        printf("%d is less than 100n",n);
        if(n%2 == 0)
            printf("%d is even",n);
        else
            printf("%d is odd",n);
    }
    else
        printf("%d is equal to or greater than 100",n);
    return 0;
}
```

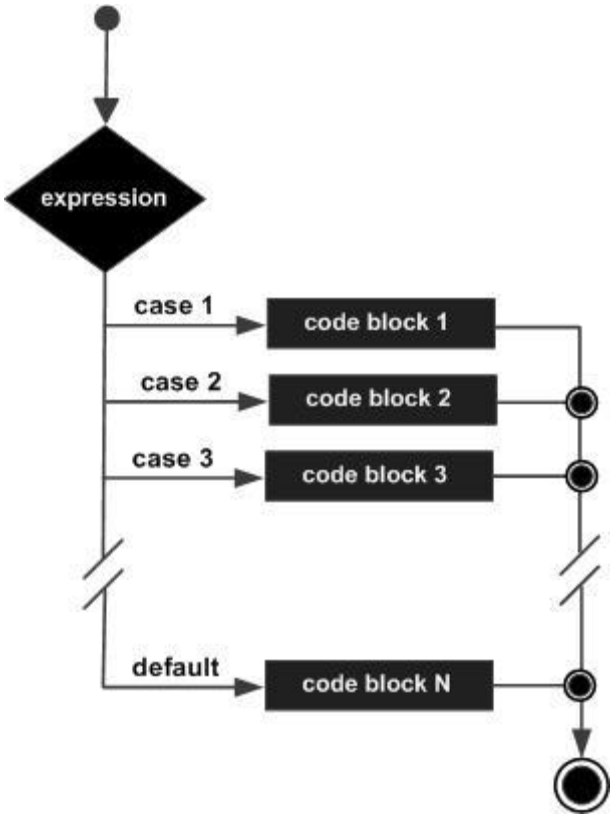
How it works:

- This program tests two conditions:
- If the number is less than 100 or not.
- If the number is less than 100 then is it odd or even.
- It consists of a nested if statement. The outer if statement checks whether the number is less than 100 or not. If the number is less than hundred, another condition i.e. if the number is even or odd is checked and respective message is displayed.

5. Switch statement:

The `switch` statement in C is a control flow statement that allows for the selection of one of many code blocks to be executed, based on the value of a single expression. It provides an alternative to using a lengthy chain of `if-else` `if-else` statements when dealing with multiple conditions based on the same variable.

- Used for multiple selections based on the value of an expression.
- Better than multiple if-else if checking the same variable.
- The expression is evaluated once, and the corresponding case block executes.
- `break` is used to exit the switch; otherwise, execution continues (fall-through).



Syntax:

```
switch(expression) {
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    default:
        // statements
}
```

Example:

```
int day = 3;
switch(day) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    case 3:
        printf("Wednesday\n");
        break;
    default:
        printf("Invalid day\n");
}
```

Switch Statement Example

```
#include <stdio.h>

int main() {
    int day = 4;

    switch(day) {
        case 1:
            printf("Monday\n");
            break;
        case 2:
            printf("Tuesday\n");
            break;
        case 3:
            printf("Wednesday\n");
            break;
        case 4:
            printf("Thursday\n");
            break;
        default:
            printf("Invalid day\n");
    }

    return 0;
}
```

How it works:

- The `day` value is evaluated once.
- The matching `case` is executed (`case 4` here).
- `break` prevents fall-through to next cases.
- If no case matches, `default` runs.

Behind the Scenes (How Decision Controls Work):

- When the program reaches a decision statement, it evaluates the condition.
- The result is either **true** (non-zero in C) or **false** (zero).
- Based on this, the program jumps to the relevant block of code.
- For `switch`, it uses the evaluated value to jump to the corresponding label.
- In assembly or machine code, these translate to conditional and unconditional jumps.

Summary:

- Decision control structures enable conditional execution.
- `if` is simple conditional checking.
- `if-else` adds alternate path.
- `if-else-if` ladder is for multiple conditions.
- `switch` is used for discrete value checking with cleaner syntax.

Introduction to Iterative Statements in C Programming

What are Iterative Statements?

- Iterative statements (also called loops) are used to execute a block of code repeatedly as long as a specified condition holds true.
- They help in automating repetitive tasks without writing the same code multiple times.

Types of Iterative Statements in C

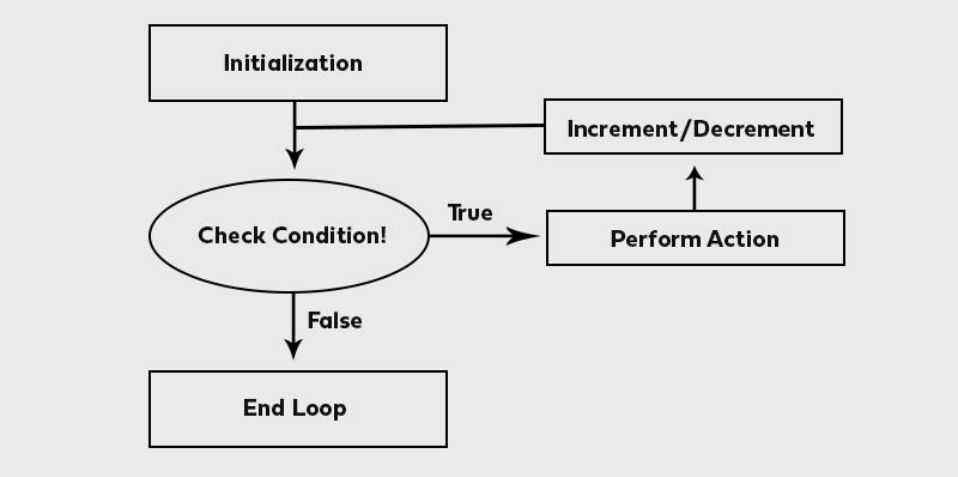
1. **while loop**
2. **do-while loop**
3. **for loop**

1. While loop:

The `while` loop in C programming is a control flow statement used to repeatedly execute a block of code as long as a specified condition remains true. It is an entry-controlled loop, meaning the condition is evaluated before the loop body is executed.

Syntax:

```
while (condition) {
    // statements to execute repeatedly
}
```



- The condition is checked **before** entering the loop body.
- If the condition is true, the loop body executes.
- Loop repeats until the condition becomes false.
- If the condition is false initially, the loop body **may never execute**.

Example:

```
int i = 1;
while (i <= 5) {
    printf("%d\n", i);
    i++;
}
```

1. while Loop Example

```
#include <stdio.h>

int main() {
    int i = 1;

    while (i <= 5) {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

How it works:

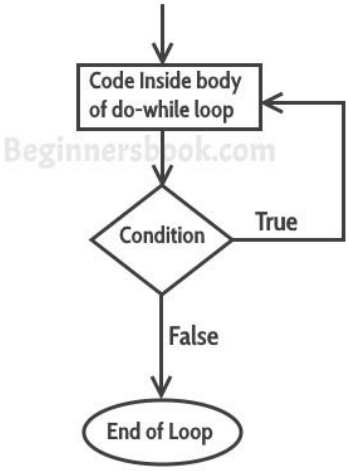
- The condition `i <= 5` is checked before the loop starts.
- If true, the loop body runs (prints `i` and increments it).
- The condition is checked again, and this repeats until `i` becomes 6 (condition false).
- If the initial condition was false, the loop body wouldn't execute.

2. do-while loop:

The `do-while` loop in C is a control flow statement that allows code to be executed repeatedly based on a given condition. It differs from the `while` loop in that the loop's body is executed at least once before the condition is evaluated.

Syntax:

```
do {
    // statements to execute
} while (condition);
```



- The loop body executes **at least once** before the condition is checked.
- After executing the body, the condition is evaluated.
- If true, loop repeats; otherwise, it stops.

Example:

```
int i = 1;
do {
    printf("%d\n", i);
    i++;
} while (i <= 5);
```

2. do-while Loop Example

```
#include <stdio.h>

int main() {
    int i = 1;

    do {
        printf("%d\n", i);
        i++;
    } while (i <= 5);

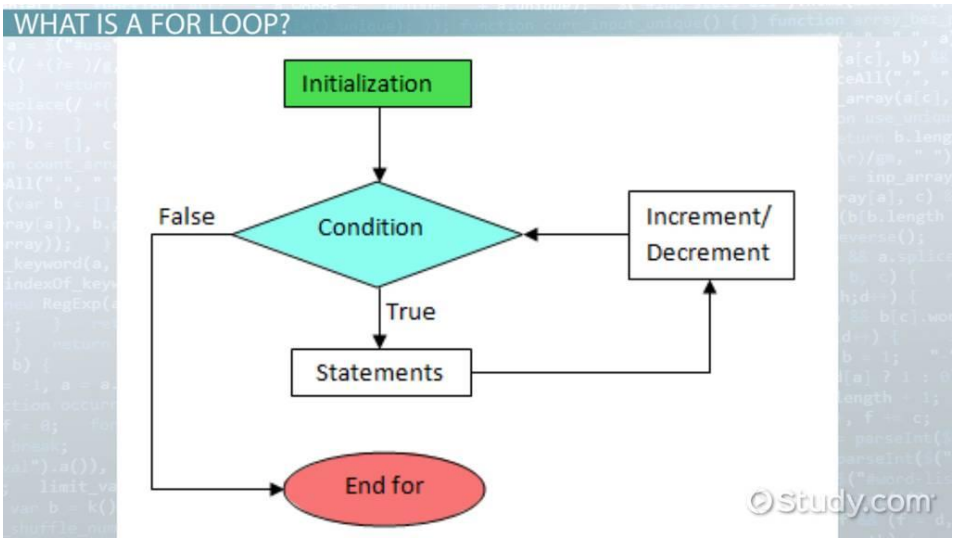
    return 0;
}
```

How it works:

- The loop body executes first (prints `i` and increments it).
- Then the condition `i <= 5` is checked.
- If true, the loop repeats; if false, it stops.
- Even if the condition is false at the start, the body executes once.

3. for loop:

The `for` loop in C is a control flow statement used to repeatedly execute a block of code a specified number of times. It is particularly useful when the number of iterations is known beforehand.



Syntax:

```
for (initialization; condition; update) {
    // statements to execute repeatedly
}
```

- Combines initialization, condition checking, and update in one line.
- Useful for counting loops or when the number of iterations is known.

Example:

```
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

for Loop Example

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        printf("%d\n", i);
    }

    return 0;
}
```

How it works:

- Initialization (`int i = 1`) runs once at the start.
- Condition (`i <= 5`) is checked before each iteration.
- If true, loop body runs (prints `i`).
- Update (`i++`) runs after each iteration.
- Loop stops when condition becomes false.

Summary:

Loop Type	Condition Checked	Executes at least once?	Best Use Case
while	Before loop body	No	When number of iterations unknown, may be zero
do-while	After loop body	Yes	When loop must run at least once
for	Before loop body	No	When number of iterations is known or controlled

---

Important Keywords in Loops

- **break**: Immediately exits the loop.
- **continue**: Skips the current iteration and jumps to the next iteration.

Introduction to Nested Loops in C Programming

What is a Nested Loop?

- A **nested loop** is a loop inside another loop.
- The **inner loop** runs completely for every single iteration of the **outer loop**.
- Useful for working with multi-dimensional data, such as matrices or tables.

Why Use Nested Loops?

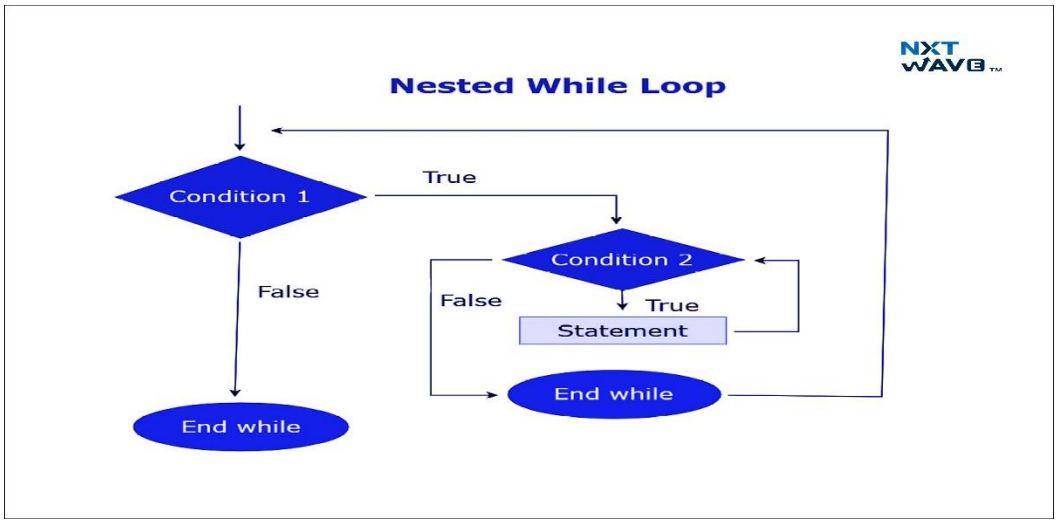
- To perform repetitive tasks within repetitive tasks.
- Common in problems involving grids, matrices, or combinations.

Syntax of Nested Loops

```
for (initialization; condition; update) {
    // Outer loop body

    for (initialization; condition; update) {
        // Inner loop body
    }
}
```

- The inner loop runs completely for every iteration of the outer loop.



Example of Nested Loops

```
#include <stdio.h>

int main() {
    int i, j;

    for (i = 1; i <= 3; i++) {          // Outer loop
        for (j = 1; j <= 2; j++) {      // Inner loop
            printf("i = %d, j = %d\n", i, j);
        }
    }

    return 0;
}
```

Output:

```
i = 1, j = 1
i = 1, j = 2
i = 2, j = 1
i = 2, j = 2
i = 3, j = 1
i = 3, j = 2
```

How Nested Loops Work:

- Outer loop starts with `i = 1`.
- Inner loop runs fully for `j = 1` to `2`.
- After inner loop finishes, outer loop increments `i` and inner loop runs again.
- Process continues until outer loop condition fails.

Practical Uses of Nested Loops

- Printing patterns (like stars, numbers).
- Traversing multi-dimensional arrays.
- Solving matrix problems (addition, multiplication).
- Generating combinations or permutations.

Important Points:

- Nested loops increase time complexity exponentially (e.g., two nested loops each running n times result in $O(n^2)$ complexity).
- Be cautious about performance when using multiple nested loops.

Pattern Printing: Right-Angled Triangle of Stars

```
#include <stdio.h>

int main() {
    int i, j;
    int rows = 5;

    for (i = 1; i <= rows; i++) {           // Outer loop for rows
        for (j = 1; j <= i; j++) {          // Inner loop for stars in each
row
            printf("* ");
        }
        printf("\n");                        // New line after each row
    }

    return 0;
}
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

Matrix Addition

```
#include <stdio.h>

int main() {
    int a[2][2] = {{1, 2}, {3, 4}};
    int b[2][2] = {{5, 6}, {7, 8}};
    int sum[2][2];
    int i, j;

    for (i = 0; i < 2; i++) {               // Outer loop for rows
        for (j = 0; j < 2; j++) {           // Inner loop for columns
            sum[i][j] = a[i][j] + b[i][j];
        }
    }

    printf("Sum of matrices:\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            printf("%d ", sum[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Sum of matrices:
6 8
10 12
```

Multiplication Table

```
#include <stdio.h>

int main() {
    int i, j;

    for (i = 1; i <= 5; i++) {              // Outer loop for numbers 1 to 5
        for (j = 1; j <= 5; j++) {          // Inner loop for multiplier 1
to 5
            printf("%d x %d = %d\t", i, j, i * j);
        }
        printf("\n");                        // New line after each table
row
    }
    return 0;
}
```

Output (formatted):

```
1 x 1 = 1    1 x 2 = 2    1 x 3 = 3    1 x 4 = 4    1 x 5 = 5
2 x 1 = 2    2 x 2 = 4    2 x 3 = 6    2 x 4 = 8    2 x 5 = 10
3 x 1 = 3    3 x 2 = 6    3 x 3 = 9    3 x 4 = 12   3 x 5 = 15
4 x 1 = 4    4 x 2 = 8    4 x 3 = 12   4 x 4 = 16   4 x 5 = 20
5 x 1 = 5    5 x 2 = 10   5 x 3 = 15   5 x 4 = 20   5 x 5 = 25
```

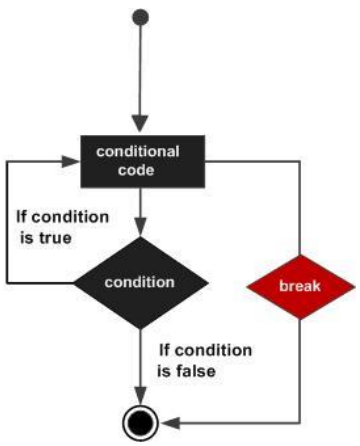
How These Examples Work:

- Outer loop controls the **rows** or **iterations**.
- Inner loop completes all its iterations for every single iteration of the outer loop.
- Together, they allow working with 2D data or multi-level repetition.

Introduction to break, continue, and goto Statements in C

1. break Statement :

The `break` statement in C programming is a jump statement used to terminate the execution of the innermost loop (for, while, do-while) or switch statement in which it appears. Upon encountering break, program control immediately transfers to the statement following the terminated loop or switch block.



- **Purpose:**
  Immediately exits the nearest enclosing loop or `switch` statement.
- **Where used:**
  Inside loops (`for`, `while`, `do-while`) or `switch`.

- **Effect:**
  Control jumps to the statement immediately after the loop or `switch`.

Example:

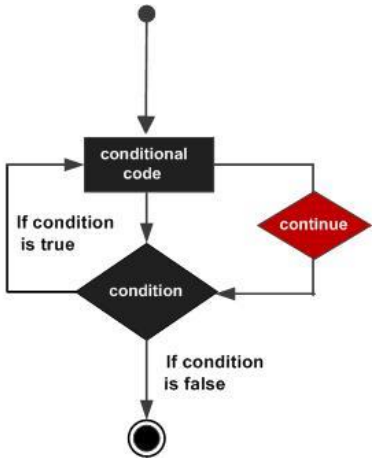
```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;   // Exit loop when i equals 5
    }
    printf("%d ", i);
}
```

- **Output:**

- `1 2 3 4`

2. continue Statement:

The `continue` statement in C is a loop control statement used to skip the remaining statements within the current iteration of a loop and proceed to the next iteration. It is particularly useful when specific conditions or scenarios within a loop require bypassing certain parts of the loop body without terminating the entire loop.



- **Purpose:**
  Skips the current iteration of the nearest enclosing loop and proceeds with the next iteration.
- **Where used:**
  Inside loops (`for`, `while`, `do-while`).
- **Effect:**
  The rest of the code inside the loop after `continue` is skipped for the current iteration.

Example:

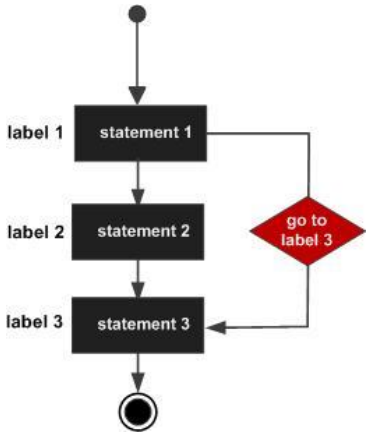
```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;  // Skip printing when i equals 3
    }
    printf("%d ", i);
}
```

Output:

```
1 2 4 5
```

3. goto Statement

The `goto` statement in C is a control flow statement that allows for an unconditional jump to a specified label within the same function. It provides a mechanism to transfer program control to a different part of the code.



- **Purpose:**
  Transfers control unconditionally to a labeled statement within the same function.
- **Where used:**
  Anywhere inside a function (use sparingly).

Syntax:

```
goto label;
...
label:
    // code here
```

- **Effect:**
  Causes an unconditional jump to the labeled statement.

Example:

```
#include <stdio.h>

int main() {
    int i = 1;
    start:
    if (i <= 5) {
        printf("%d ", i);
        i++;
        goto start;  // Loop back to 'start' label
    }
    return 0;
}
```

- **Output:**
  `1 2 3 4 5`
- **Caution:**
  Overusing `goto` can make code confusing and hard to maintain ("spaghetti code").
  Generally avoided except for error handling or breaking out of deeply nested loops.

Summary Table

Statement	Purpose	Used In	Effect
`break`	Exit loop or switch immediately	loops, switch	Control jumps after the loop or switch
`continue`	Skip current loop iteration	loops	Skips rest of loop body, starts next iteration
`goto`	Unconditional jump to label	anywhere in function	Control jumps to labeled statement